Experiment 2: Create Login/Signup System with Flask and MongoDB

Aim

To build a complete user authentication system with registration and login functionality using Flask backend and MongoDB Atlas cloud database.

Software Requirements

- Python 3.10+
- Flask
- pymongo
- MongoDB Atlas account (free tier)
- bcrypt (for password hashing)

Theory

User authentication is a fundamental feature of web applications. It involves verifying user identity through credentials (email/password). MongoDB Atlas is a cloud-hosted NoSQL database service that provides scalability and flexibility. We use bcrypt to hash passwords for security, ensuring that even if the database is compromised, passwords remain protected.

Procedure

Step 1: Create Project Directory

```
mkdir exp2_auth_system cd
exp2_auth_system
```

Step 2: Create and Activate Virtual Environment

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

Step 3: Install Required Packages

```
pip install flask pymongo bcrypt python-dotenv
```

Step 4: Set Up MongoDB Atlas

1. Go to https://www.mongodb.com/cloud/atlas/register
2. Create a free account
3. Click "Build a Database" → Select "Free" (M0)
4. Choose a cloud provider (AWS/Google Cloud/Azure) and region
5. Click "Create Cluster"
6. Under Security → Database & Network Access:
    - Click "Add New Database User"
    - Choose "Password" authentication
    - Username: `admin`
    - Password: Create a strong password (save it!)
    - User Privileges: "Atlas admin"
    - Click "Add User"
7. Under Security → Database & Network Access:
    - Click "IP Access List"
    - Click "Add IP Address"
    - Click "Allow Access from Anywhere" (0.0.0.0/0)
    - Click "Confirm"
8. Go to Database → "Cluster" → Click "Connect"
9. Under "Connect your application", choose "Drivers"
10. Copy the connection string (looks like: `mongodb+srv://admin:<password>@cluster0.xxxxx.mongodb.net/`)
11. Replace `<password>` with your actual password

Step 5: Create `.env` File

Create `.env` in the root directory.

```
MONGO_URI=mongodb+srv://admin:YOUR_PASSWORD@cluster0.xxxxx.mongodb.net/
SECRET_KEY=your_secret_key_here_make_it_random_and_long
```

Replace `YOUR_PASSWOR` with your MongoDB password.

> **Note**
>
> Replace the right side part of above `.env` with actual connection string you copied before. `MONGO_URI` `<paste copied connection string here>`

> **Warning**
>
> Make sure to keep the `.env` file private. DO NOT PUSH to GitHub, if configured.

Step 6: Create Project Structure

```
mkdir templates
```

Step 7: Create app.py

```
from flask import Flask, render_template, request, redirect, url_for, flash, session from pymongo
import MongoClient
import bcrypt
import os
from dotenv import load_dotenv

# Load environment variables load_dotenv()

app = Flask(__name__)
app.secret_key = os.getenv('SECRET_KEY', 'default_fallback_secret_key')

# MongoDB connection
client = MongoClient(os.getenv('MONGO_URI'))
db = client['fullstack_ai_lab'] users_collection =
db['users']
@app.route('/')
def home():
    if 'email' in session:
        return render_template('dashboard.html', email=session['email']) return
    redirect(url_for('login'))

@app.route('/signup', methods=['GET', 'POST']) def
signup():
    if request.method == 'POST':
        email = request.form.get('email') password =
        request.form.get('password')
        confirm_password = request.form.get('confirm_password')

        # Validation
        if not email or not password:
            flash('Email and password are required!', 'error') return
            redirect(url_for('signup'))

        if password != confirm_password: flash('Passwords
        do not match!', 'error') return
        redirect(url_for('signup'))
         # Check if user already exists
        existing_user = users_collection.find_one({'email': email}) if
        existing_user:
            flash('Email already registered!', 'error') return
            redirect(url_for('signup'))
        # Hash password
        hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())

        # Insert user into database
        users_collection.insert_one({
            'email': email,
```

```
            'password': hashed_password
        })

        flash('Registration successful! Please login.', 'success') return
        redirect(url_for('login'))

    return render_template('signup.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form.get('email')
        password =
        request.form.get('password')

        # Find user in database
        user = users_collection.find_one({'email': email})

        if user and bcrypt.checkpw(password.encode('utf-8'), user['password']): session['email']
            = email
            flash('Login successful!', 'success') return
            redirect(url_for('home'))
        else:
            flash('Invalid email or password!', 'error')
            return redirect(url_for('login'))

    return render_template('login.html')

@app.route('/logout')
def logout():
    session.pop('email', None)
    flash('Logged out successfully!', 'success') return
    redirect(url_for('login'))

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Step 8: Create templates/signup.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sign Up - Full Stack AI Lab</title>
</head>
<body>
    <div class="auth-container">
        <h1>Create Account</h1>

        {% with messages = get_flashed_messages(with_categories=true) %}
            {% if messages %}
                {% for category, message in messages %}
                    <div class="alert alert-{{ category }}">{{ message }}</div>
```

```
                {% endfor %}
            {% endif %}
        {% endwith %}

        <form method="POST" action="/signup">
            <div class="form-group">
                <label for="email">Email Address</label>
                <input type="email" id="email" name="email" required>
            </div>

            <div class="form-group">
                <label for="password">Password</label>
                <input type="password" id="password" name="password" required>
            </div>

            <div class="form-group">
                <label for="confirm_password">Confirm Password</label>
                <input type="password" id="confirm_password" name="confirm_password"
required>
            </div>

            <button type="submit" class="btn">Sign Up</button>
        </form>

        <p class="switch-auth">Already have an account? <a href="/login">Login here</a></p>
    </div>
</body>
</html>
```

Step 9: Create templates/login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login - Full Stack AI Lab</title>
</head>
<body>
    <div class="auth-container">
        <h1>Login</h1>

        {% with messages = get_flashed_messages(with_categories=true) %}
            {% if messages %}
                {% for category, message in messages %}
                    <div class="alert alert-{{ category }}">{{ message }}</div>
                {% endfor %}
            {% endif %}
        {% endwith %}

        <form method="POST" action="/login">
            <div class="form-group">
                <label for="email">Email Address</label>
```

```
                <input type="email" id="email" name="email" required>
            </div>

            <div class="form-group">
                <label for="password">Password</label>
                <input type="password" id="password" name="password" required>
            </div>

            <button type="submit" class="btn">Login</button>
        </form>

        <p class="switch-auth">Don't have an account? <a href="/signup">Sign up here</a></p>
    </div>
</body>
</html>
```

Step 10: Create templates/dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dashboard - Full Stack AI Lab</title>
</head>
<body>
    <div class="auth-container">
        <h1>Welcome to Dashboard</h1>
        <p>You are logged in as: <strong>{{ email }}</strong></p>

        {% with messages = get_flashed_messages(with_categories=true) %}
            {% if messages %}
                {% for category, message in messages %}
                    <div class="alert alert-{{ category }}">{{ message }}</div>
                {% endfor %}
            {% endif %}
        {% endwith %}

        <a href="/logout" class="btn">Logout</a>
    </div>
</body>
</html>
```

Step 11: Create requirements.txt

```
pip freeze > requirements.txt
```

Step 12: Run the Application

```
python app.py
```

Step 13: Test the Application

1. Go to `http://localhost:5000/signup`
2. Register with an email and password
3. Login with your credentials
4. Verify you're redirected to the dashboard
5. Check MongoDB Atlas to see your user record

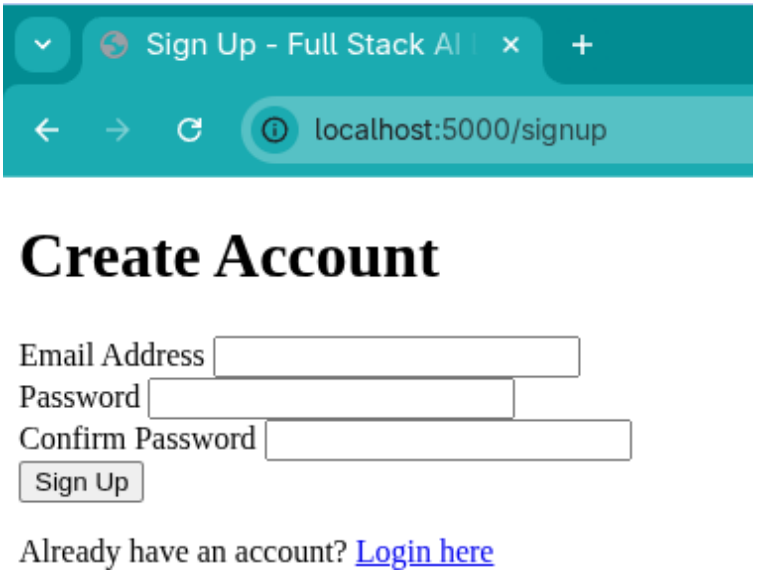
Output

A complete authentication system with:

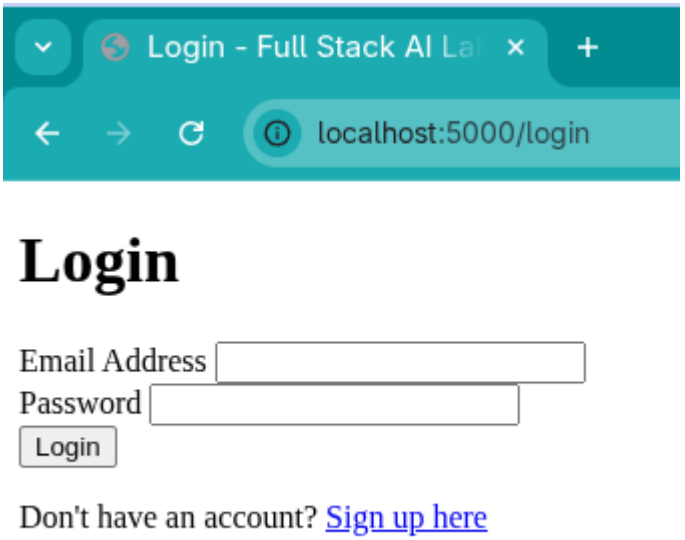
- User registration with password confirmation
- Password hashing for security
- Login functionality with session management
- Flash messages for user feedback
- User data stored in MongoDB Atlas
- Protected dashboard page

Screen-shots

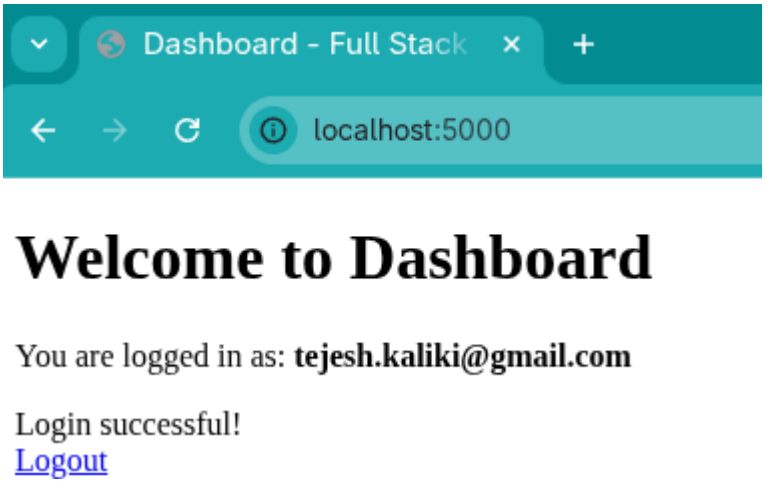
SignUp Page:



Login Page:



Logged in Page:



User created in MongoDB Atlass:

```
_id: ObjectId('6919f5ef1a2d1a99ab06ea22')
email : "tejesh.kaliki@gmail.com"
password : Binary.createFromBase64('JDJiJDEyJHA3Vm9QY05mbERnaFNZem9ZUy9Qek9Gd2J2ekRBRW9KU1k1MUJKZVJyUmE1cEIySi84Ni5h', 0)
```

Result

Successfully implemented a secure authentication system using Flask and MongoDB Atlas with password hashing and session management.